МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №8

"Розробка REST API на FastAPI та асинхронного клієнта"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Варіант 1. Система обліку студентів. Мета роботи: розробити повнофункціональний REST API для системи обліку студентів за допомогою FastAPI, Pydantic v2 та SQLAlchemy ORM. Реалізувати валідацію схем, обробку винятків, транзакційні сесії для кожного запиту, пагінацію, фільтрацію, статистику та асинхронний клієнт на базі httpx із підтримкою повторних спроб і семафора обмеження конкурентності.

Теоретичні відомості:

FastAPI побудований на базі Starlette та Pydantic, що забезпечує автоматичну валідацію вхідних даних, генерацію інтерактивної документації OpenAPI (Swagger) та підтримку асинхронних ендпоінтів. Управління транзакціями бази даних на рівні запиту реалізується за допомогою залежностей (Depends) та контекстних менеджерів, гарантуючи ізольованість і своєчасний rollback у разі виникнення помилок. Асинхронні HTTP-клієнти (httpx) дозволяють ефективно виконувати масові паралельні виклики з контролем навантаження через asyncio.Semaphore.

Виконання роботи

1. Варіант 1: постановка задачі, предметна область та схеми даних Pydantic

```python
from __future__ import annotations

import re
from typing import Annotated, Literal, Self

from pydantic import (
    BaseModel,
    ConfigDict,
    Field,
    field_validator,
    model_validator,
)

from student_manager.persistence.models import StudentDTO, StudentRow

EMAIL_REGEX = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
SQLITE_MAX_INT: int = 9_223_372_036_854_775_807


class GroupCreate(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    name: Annotated[str, Field(min_length=1, max_length=100)]


class GroupResponse(BaseModel):
    model_config = ConfigDict(
        extra="forbid", from_attributes=True, str_strip_whitespace=True
    )
    id: Annotated[int, Field(strict=True, gt=0, le=SQLITE_MAX_INT)]
    name: Annotated[str, Field(min_length=1, max_length=100)]


class StudentCreate(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    first_name: Annotated[str, Field(min_length=1, max_length=100)]
```

```
    last_name: Annotated[str, Field(min_length=1, max_length=100)]
    group_id: Annotated[int, Field(strict=True, gt=0, le=SQLITE_MAX_INT)]
    average_grade: Annotated[
        float, Field(strict=True, allow_inf_nan=False, ge=0.0, le=100.0)
    ]
    student_id: Annotated[
        int | None, Field(default=None, strict=True, gt=0, le=SQLITE_MAX_INT)
    ] = None
    email: str | None = None

    @field_validator("email")
    @classmethod
    def validate_email(cls, v: str | None) -> str | None:
        if v is None:
            return None
        cleaned = v.strip()
        if not cleaned:
            return None
        if not EMAIL_REGEX.match(cleaned):
            raise ValueError(f"Invalid email format: '{cleaned}'")
        return cleaned


class StudentPatch(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    first_name: Annotated[
        str | None, Field(default=None, min_length=1, max_length=100)
    ] = None
    last_name: Annotated[
        str | None, Field(default=None, min_length=1, max_length=100)
    ] = None
    group_id: Annotated[
        int | None, Field(default=None, strict=True, gt=0, le=SQLITE_MAX_INT)
    ] = None
    average_grade: Annotated[
```

```
        float | None,
        Field(default=None, strict=True, allow_inf_nan=False, ge=0.0, le=100.0),
    ] = None
    email: str | None = None

    @field_validator("email")
    @classmethod
    def validate_email(cls, v: str | None) -> str | None:
        if v is None:
            return None
        cleaned = v.strip()
        if not cleaned:
            return None
        if not EMAIL_REGEX.match(cleaned):
            raise ValueError(f"Invalid email format: '{cleaned}'")
        return cleaned

    @model_validator(mode="after")
    def validate_patch_non_empty(self) -> Self:
        fields_set = self.model_fields_set
        if not fields_set:
            raise ValueError("Patch payload cannot be empty")
        for field_name in ("first_name", "last_name", "group_id", "average_grade"):
            if field_name in fields_set and getattr(self, field_name) is None:
                raise ValueError(f"Field '{field_name}' cannot be explicitly null")
        return self


class StudentResponse(BaseModel):
    model_config = ConfigDict(
        extra="forbid", from_attributes=True, str_strip_whitespace=True
    )
    student_id: Annotated[int, Field(strict=True, gt=0, le=SQLITE_MAX_INT)]
    first_name: Annotated[str, Field(min_length=1, max_length=100)]
    last_name: Annotated[str, Field(min_length=1, max_length=100)]
```

```
    group_id: Annotated[int, Field(strict=True, gt=0, le=SQLITE_MAX_INT)]
    group: Annotated[str, Field(min_length=1, max_length=100)]
    average_grade: Annotated[
        float, Field(strict=True, allow_inf_nan=False, ge=0.0, le=100.0)
    ]
    email: str | None = None

    @classmethod
    def from_dto_or_row(cls, obj: StudentDTO | StudentRow) -> StudentResponse:
        return cls(
            student_id=obj.id,
            first_name=obj.first_name,
            last_name=obj.last_name,
            group_id=obj.group_id,
            group=obj.group_name if isinstance(obj, StudentDTO) else obj.group.name,
            average_grade=obj.average_grade,
            email=obj.email,
        )


class GroupStatistics(BaseModel):
    model_config = ConfigDict(extra="forbid")
    group_id: Annotated[int, Field(strict=True, gt=0, le=SQLITE_MAX_INT)]
    count: Annotated[int, Field(strict=True, ge=0, le=SQLITE_MAX_INT)]
    average: Annotated[
        float | None, Field(strict=True, allow_inf_nan=False, ge=0.0, le=100.0)
    ] = None
    best_student: StudentResponse | None = None


class HealthResponse(BaseModel):
    model_config = ConfigDict(extra="forbid")
    status: Literal["ok"]
    environment: str
```

2. Реалізація маршрутів FastAPI та керування транзакційними сесіями

```
from __future__ import annotations

import os
import threading
from collections.abc import AsyncIterator, Generator
from contextlib import asynccontextmanager
from typing import Any

from alembic import command
from alembic.config import Config
from fastapi import Depends, FastAPI, HTTPException, Path, Query, Request, Response
from fastapi.responses import JSONResponse
from sqlalchemy import event, text
from sqlalchemy.engine import make_url
from sqlalchemy.exc import IntegrityError, SQLAlchemyError
from sqlalchemy.orm import Session
from sqlalchemy.pool import StaticPool

from student_manager.api_schemas import (
    SQLITE_MAX_INT,
    GroupCreate,
    GroupResponse,
    GroupStatistics,
    HealthResponse,
    StudentCreate,
    StudentPatch,
    StudentResponse,
)
from student_manager.models import Student
from student_manager.persistence.database import (
    create_database_engine,
    create_session_factory,
)
from student_manager.persistence.service import StudentPersistenceService
```

```python
def get_session(request: Request) -> Generator[Session, None, None]:
    """Mutations commit before responding; reads and failed requests roll back."""
    lock = getattr(request.app.state, "db_lock", None)
    if lock is not None:
        lock.acquire()
    session = request.app.state.session_factory()
    try:
        session.begin()
        if (
            request.method in {"POST", "PATCH", "PUT", "DELETE"}
            and request.app.state.engine.dialect.name == "sqlite"
        ):
            session.connection(execution_options={"sqlite_immediate": True})
        yield session
        if request.method in {"POST", "PATCH", "PUT", "DELETE"}:
            session.commit()
        else:
            session.rollback()
    except BaseException:
        session.rollback()
        raise
    finally:
        try:
            session.close()
        finally:
            if lock is not None:
                lock.release()


SESSION_DEPENDENCY = Depends(get_session, scope="function")


def _student(service: StudentPersistenceService, student_id: int) -> StudentResponse:
    dto = service.get_student_dto(student_id)
```

```python
    if dto is None:
        raise HTTPException(404, "Student not found")
    return StudentResponse.from_dto_or_row(dto)


def create_app(database_url: str | None = None) -> FastAPI:
    @asynccontextmanager
    async def lifespan(application: FastAPI) -> AsyncIterator[None]:
        url = (
            database_url
            if database_url is not None
            else os.environ.get("DATABASE_URL", "sqlite:///students.sqlite")
        )
        parsed = make_url(url)
        options: dict[str, Any] = {}
        if parsed.get_backend_name() == "sqlite":
            options["connect_args"] = {"check_same_thread": False, "timeout": 30.0}
            if parsed.database in (None, "", ":memory:"):
                options["poolclass"] = StaticPool
        engine = create_database_engine(url, **options)
        application.state.engine = engine
        application.state.db_lock = (
            threading.Lock() if isinstance(engine.pool, StaticPool) else None
        )
        try:
            if engine.dialect.name == "sqlite":
                # без явного begin sqlite може закомітити savepoint раніше за запит
                # стартуєм як immediate шоб не ловити upgrade deadlock

                @event.listens_for(engine, "connect")
                def explicit_transactions(connection: Any, record: Any) -> None:
                    connection.isolation_level = None
                    cursor = connection.cursor()
                    cursor.execute("PRAGMA busy_timeout = 30000")
                    if parsed.database not in (None, "", ":memory:"):
```

```
                    cursor.execute("PRAGMA journal_mode = WAL")
                cursor.close()

            @event.listens_for(engine, "begin")
            def begin(connection: Any) -> None:
                if connection.get_execution_options().get(
                    "sqlite_immediate", False
                ):
                    connection.exec_driver_sql("BEGIN IMMEDIATE")
                else:
                    connection.exec_driver_sql("BEGIN")

        config = Config()
        config.set_main_option(
            "script_location", "student_manager.persistence:migrations"
        )
        with engine.connect() as connection:
            config.attributes["connection"] = connection
            command.upgrade(config, "head")
        application.state.session_factory = create_session_factory(engine)
        yield
    finally:
        engine.dispose()

application = FastAPI(
    title="Student Manager API", version="0.8.0", lifespan=lifespan
)

@application.exception_handler(IntegrityError)
async def conflict(request: Request, exc: IntegrityError) -> JSONResponse:
    return JSONResponse(status_code=409, content={"detail": "Resource conflict"})

@application.exception_handler(ValueError)
async def invalid_value(request: Request, exc: ValueError) -> JSONResponse:
    return JSONResponse(
```

```
        status_code=422, content={"detail": "Invalid resource data"}
    )

@application.exception_handler(OverflowError)
async def integer_overflow(request: Request, exc: OverflowError) -> JSONResponse:
    return JSONResponse(status_code=422, content={"detail": "Integer overflow"})

@application.exception_handler(Exception)
async def internal_error(request: Request, exc: Exception) -> JSONResponse:
    return JSONResponse(
        status_code=500, content={"detail": "Internal server error"}
    )

@application.get("/health", response_model=HealthResponse)
def health(request: Request) -> HealthResponse:
    lock = getattr(request.app.state, "db_lock", None)
    try:
        if lock is not None:
            with lock, request.app.state.engine.connect() as connection:
                connection.execute(text("SELECT 1"))
        else:
            with request.app.state.engine.connect() as connection:
                connection.execute(text("SELECT 1"))
    except SQLAlchemyError:
        raise HTTPException(503, "Database unavailable") from None
    return HealthResponse(
        status="ok", environment=os.getenv("ENVIRONMENT", "development")
    )

@application.post("/groups", response_model=GroupResponse, status_code=201)
def create_group(
    data: GroupCreate, session: Session = SESSION_DEPENDENCY
) -> GroupResponse:
    return GroupResponse.model_validate(
        StudentPersistenceService(session=session).create_group(data.name)
```

```
        )

    @application.get("/groups", response_model=list[GroupResponse])
    def list_groups(session: Session = SESSION_DEPENDENCY) -> list[GroupResponse]:
        return [
            GroupResponse.model_validate(group)
            for group in StudentPersistenceService(session=session).list_groups()
        ]

    @application.post("/students", response_model=StudentResponse, status_code=201)
    def create_student(
        data: StudentCreate, session: Session = SESSION_DEPENDENCY
    ) -> StudentResponse:
        service = StudentPersistenceService(session=session)
        group = service.get_group(data.group_id)
        if group is None:
            raise HTTPException(404, "Group not found")
        student = Student(
            data.first_name,
            data.last_name,
            group.name,
            data.average_grade,
            data.student_id,
        )
        created = service.create_student(
            student, group_id=data.group_id, email=data.email
        )
        assert created.student_id is not None
        return _student(service, created.student_id)

    @application.get("/students/{student_id}", response_model=StudentResponse)
    def get_student(
        student_id: int = Path(gt=0, le=SQLITE_MAX_INT),
        session: Session = SESSION_DEPENDENCY,
    ) -> StudentResponse:
```

3. Реалізація асинхронного клієнта групової статистики з семафором та retries

```
import asyncio
import math
import urllib.parse
from collections.abc import Sequence
from typing import cast

import httpx


def _validate_options(
    group_ids: Sequence[int],
    base_url: str,
    timeout: float,
    retries: int,
    concurrency: int,
    backoff: float,
    client: httpx.AsyncClient | None,
) -> None:
    # перевіряємо параметри виклику
    if not isinstance(group_ids, Sequence) or isinstance(
        group_ids, (str, bytes, bytearray)
    ):
        raise TypeError("group_ids must be a sequence of positive integers")

    for gid in group_ids:
        if isinstance(gid, bool) or not isinstance(gid, int):
            raise TypeError(f"group_id must be an integer, got {gid!r}")
        if gid <= 0:
            raise ValueError(f"group_id must be positive, got {gid!r}")

    if not isinstance(base_url, str):
        raise TypeError("base_url must be a string")
    if not base_url.strip():
        raise ValueError("base_url must be a non-empty string")
```

```
    parsed = urllib.parse.urlsplit(base_url)
    if parsed.scheme not in ("http", "https") or not parsed.netloc:
        raise ValueError(f"base_url must be a valid http/https URL, got {base_url!r}")

    if isinstance(timeout, bool) or not isinstance(timeout, (int, float)):
        raise TypeError("timeout must be a number")
    if timeout <= 0:
        raise ValueError("timeout must be a positive number")

    if isinstance(retries, bool) or not isinstance(retries, int):
        raise TypeError("retries must be an integer")
    if retries < 0:
        raise ValueError("retries must be a non-negative integer")

    if isinstance(concurrency, bool) or not isinstance(concurrency, int):
        raise TypeError("concurrency must be an integer")
    if concurrency < 1:
        raise ValueError("concurrency must be an integer >= 1")

    if isinstance(backoff, bool) or not isinstance(backoff, (int, float)):
        raise TypeError("backoff must be a number")
    if backoff < 0:
        raise ValueError("backoff must be a non-negative number")

    if client is not None and not isinstance(client, httpx.AsyncClient):
        raise TypeError("client must be an httpx.AsyncClient instance or None")


def _validate_best_student(student: object) -> None:
    if student is None:
        return
    if not isinstance(student, dict):
        raise TypeError("Invalid best_student: expected StudentResponse or None")

    required_keys = {
```

```
        "student_id",
        "first_name",
        "last_name",
        "group_id",
        "group",
        "average_grade",
    }
    if not required_keys.issubset(student) or student.keys() - required_keys - {
        "email"
    }:
        raise ValueError("Invalid best_student fields")

    for field in ("student_id", "group_id"):
        value = student[field]
        if isinstance(value, bool) or not isinstance(value, int):
            raise TypeError(f"Invalid best_student {field}: expected integer")
        if value <= 0:
            raise ValueError(f"Invalid best_student {field}: must be positive")

    for field in ("first_name", "last_name", "group"):
        value = student[field]
        if not isinstance(value, str):
            raise TypeError(f"Invalid best_student {field}: expected string")
        if not value.strip():
            raise ValueError(f"Invalid best_student {field}: must not be blank")

    grade = student["average_grade"]
    if isinstance(grade, bool) or not isinstance(grade, (int, float)):
        raise TypeError("Invalid best_student average_grade: expected number")
    if not 0 <= grade <= 100:
        raise ValueError(
            "Invalid best_student average_grade: must be finite in [0, 100]"
        )

    email = student.get("email")
```

```
    if email is not None and not isinstance(email, str):
        raise TypeError("Invalid best_student email: expected string or None")


def _validate_schema(data: object, expected_group_id: int) -> dict[str, object]:
    # валідуємо структуру відповіді з сервера
    if not isinstance(data, dict):
        raise TypeError(
            f"Expected dict response for group {expected_group_id}, got {type(data).__name__}"
        )

    required_keys = {"group_id", "count", "average", "best_student"}
    if not required_keys.issubset(data.keys()):
        missing = required_keys - set(data.keys())
        raise ValueError(
            f"Missing required fields {missing} in group {expected_group_id} statistics"
        )

    gid = data["group_id"]
    if isinstance(gid, bool) or not isinstance(gid, int):
        raise TypeError(f"Invalid group_id type in response: {type(gid).__name__}")
    if gid != expected_group_id:
        raise ValueError(
            f"Mismatched group_id in response: {gid!r}, expected {expected_group_id}"
        )

    count = data["count"]
    if isinstance(count, bool) or not isinstance(count, int):
        raise TypeError(f"Invalid count type in response: {type(count).__name__}")
    if count < 0:
        raise ValueError(f"Invalid count in response: {count!r}")

    average = data["average"]
    if average is not None and (
        isinstance(average, bool) or not isinstance(average, (int, float))
    ):
        raise TypeError(f"Invalid average type in response: {type(average).__name__}")
    if isinstance(average, float) and not math.isfinite(average):
        raise ValueError("Invalid average in response: must be finite")

    _validate_best_student(data["best_student"])

    # вертаємо типізований словник зі збереженням значень
    return cast(dict[str, object], data)


async def _fetch_single_group(
    client: httpx.AsyncClient,
    base_url: str,
    group_id: int,
    semaphore: asyncio.Semaphore,
    timeout: float,
    retries: int,
    backoff: float,
) -> dict[str, object]:
    # робимо запит по окремій групі з обмеженням семафором та повторами
    url = f"{base_url.rstrip('/')}/groups/{group_id}/statistics"

    for attempt in range(retries + 1):
        try:
            async with semaphore:
                response = await client.get(url, timeout=timeout)
        except (httpx.TimeoutException, httpx.NetworkError):
            if attempt < retries:
                delay = min(backoff * (2**attempt), 10.0)
                await asyncio.sleep(delay)
                continue
            raise

        status = response.status_code
```

```python
        if 200 <= status < 300:
            try:
                payload = cast(object, response.json())
            except Exception as exc:
                # битий json не повторюємо
                raise ValueError(
                    f"Malformed JSON response for group {group_id}"
                ) from exc

            return _validate_schema(payload, group_id)

        if status == 429 or (500 <= status < 600):
            # тимчасові помилки повторюємо
            if attempt < retries:
                delay = min(backoff * (2**attempt), 10.0)
                await asyncio.sleep(delay)
                continue
            response.raise_for_status()

        # постійні помилки 4xx або інші кидаємо негайно
        response.raise_for_status()

    # захисний випадок якщо вичерпались спроби
    raise RuntimeError(f"Retries exhausted for group {group_id}")


async def fetch_group_statistics(
    group_ids: Sequence[int],
    *,
    base_url: str,
    client: httpx.AsyncClient | None = None,
    timeout: float = 5.0,
    retries: int = 2,
    concurrency: int = 4,
    backoff: float = 0.05,
```

```
) -> list[dict[str, object]]:
    # завантажує статистику груп паралельно з контролем одночасності
    _validate_options(
        group_ids,
        base_url,
        timeout,
        retries,
        concurrency,
        backoff,
        client,
    )

    if not group_ids:
        return []

    owned_client = client is None
    http_client = client if client is not None else httpx.AsyncClient()
    semaphore = asyncio.Semaphore(concurrency)

    tasks: list[asyncio.Task[dict[str, object]]] = []
    try:
        for gid in group_ids:
            task = asyncio.create_task(
                _fetch_single_group(
                    http_client,
                    base_url,
                    gid,
                    semaphore,
                    timeout,
                    retries,
                    backoff,
                )
            )
            tasks.append(task)
```

```
        results = await asyncio.gather(*tasks)
        return list(results)
    except BaseException:
        # при помилці чи скасуванні скасовуємо залишок задач
        for task in tasks:
            if not task.done():
                task.cancel()
        if tasks:
            await asyncio.gather(*tasks, return_exceptions=True)
        raise
    finally:
        # закриваємо тільки власний клієнт
        if owned_client:
            await http_client.aclose()
```

4. Результати реального виконання запитів до локального сервера API

	URL / Ендпоінт	Статус	Опис результату
	64802/health		startup health
	64802/health		/health
	64802/openapi.json		/openapi.json
	64802/students		/students
	64802/groups		/groups

	64802/groups		/groups
	64802/groups		/groups
	64802/groups		/groups
	64802/groups		/groups
	64802/students		/students
	64802/students/8		/students/8
	64802/students		/students

5. Автоматизоване тестування API, схем та асинхронного клієнта

```
PS C:\Users\ponti> pytest tests/test_api.py tests/test_api_schemas.py tests/test_group_client.py -v
tests/test_api.py::test_health_endpoint PASSED
tests/test_api.py::test_create_and_get_student PASSED
tests/test_api.py::test_file_sqlite_concurrent_writes_and_upgrades PASSED
tests/test_api.py::test_memory_sqlite_static_pool_concurrent_requests PASSED
tests/test_api.py::test_extremely_large_integer_ids_return_422 PASSED
tests/test_api_schemas.py::test_schemas_reject_integer_overflow PASSED
tests/test_group_client.py::test_fetch_group_statistics_success PASSED
tests/test_group_client.py::test_fetch_group_statistics_retry_on_failure PASSED

======================== 126 passed, 1 warning in 7.06s ========================
PS C:\Users\ponti> _
```

6. Відповіді на контрольні запитання

Питання: Що таке комп'ютерна мережа?

Відповідь: Комп'ютерна мережа - це сукупність з'єднаних між собою обчислювальних пристроїв, які обмінюються даними та спільними ресурсами за допомогою визначених протоколів зв'язку. Вона забезпечує взаємодію між серверами, робочими станціями та мережевими службами.

Питання: Що таке клієнт-серверна архітектура?

Відповідь: Клієнт-серверна архітектура - це модель мережевої взаємодії, де клієнт надсилає запити на отримання ресурсів або виконання операцій, а сервер обробляє їх і повертає відповідь. Це розділяє інтерфейс користувача та бізнес-логіку або зберігання даних.

Питання: Що таке HTTP?

Відповідь: HTTP (HyperText Transfer Protocol) - це протокол прикладного рівня для передавання структурованих гіпертекстових та довільних двійкових або текстових даних у комп'ютерних мережах. Він функціонує за моделлю запит-відповідь поверх надійного транспортного протоколу TCP.

Питання: Що містить HTTP request?

Відповідь: HTTP request містить метод запиту, URL ресурсу, версію протоколу, набір заголовків (headers) та опціональне тіло запиту (body). Заголовки передають метадані, такі як формат контенту та параметри автентифікації, а тіло містить корисне навантаження.

Питання: Що містить HTTP response?

Відповідь: HTTP response складається зі статусного рядка з числовим кодом статусу та описом, заголовків відповіді й опціонального тіла (body). Тіло відповіді зазвичай містить запитаний ресурс, наприклад JSON-документ або HTML-сторінку.

Питання: Для чого використовується GET?

Відповідь: Метод GET використовується для безпечного та ідемпотентного отримання ресурсів із сервера без модифікації його стану. Він передає параметри фільтрації чи пагінації у рядку запиту (query parameters) та зазвичай не має тіла запиту.

Питання: Для чого використовується POST?

Відповідь: Метод POST використовується для надсилання даних на сервер з метою створення нового підпорядкованого ресурсу або виклику обробки. Він не є ідемпотентним і передає основне навантаження всередині тіла запиту.

Питання: Для чого використовується PUT?

Відповідь: Метод PUT використовується для повного оновлення або створення зазначеного за точним URI ресурсу переданим у тілі запиту вмістом. Цей метод є ідемпотентним, тобто багаторазове повторення однакового запиту призводить до однакового стану ресурсу на сервері.

Питання: Для чого використовується PATCH?

Відповідь: Метод PATCH використовується для часткової модифікації існуючого ресурсу, передаючи лише змінені поля замість повної заміни сутності. На відміну від PUT, він мінімізує обсяг переданих мережею даних при зміні окремих атрибутів.

Питання: Для чого використовується DELETE?

Відповідь: Метод DELETE використовується для видалення вказаного за ідентифікатором або URI ресурсу на сервері. Цей метод є ідемпотентним, оскільки після першого успішного видалення повторний запит не змінює кінцевий стан відсутності ресурсу.

Питання: Що означає status code 200?

Відповідь: Статус-код 200 OK означає, що запит клієнта успішно отримано, зрозуміло та оброблено сервером. Для GET-запитів він зазвичай повертає запитані дані у тілі відповіді.

Питання: Що означає 201?

Відповідь: Статус-код 201 Created сигналізує про успішне створення нового ресурсу на сервері в результаті обробки запиту (зазвичай POST). Разом із ним зазвичай повертається створений об'єкт та заголовок Location із його адресою.

Питання: Що означає 204?

Відповідь: Статус-код 204 No Content означає успішну обробку запиту сервером за відсутності вмісту в тілі відповіді. Його часто використовують для операцій DELETE або оновлень, коли клієнту достатньо лише підтвердження успіху.

Питання: Що означає 404?

Відповідь: Статус-код 404 Not Found вказує на те, що сервер не знайшов ресурс за запитаним URI. Це означає неіснуючий кінцевий маршрут або відсутність запису з відповідним ідентифікатором у базі даних.

Питання: Що означає 422?

Відповідь: Статус-код 422 Unprocessable Entity сигналізує, що синтаксис запиту правильний, але вміст не відповідає семантичним або валідаційним правилам. У FastAPI та Pydantic цей код автоматично повертається у разі невідповідності типів чи обмежень вхідних полів.

Питання: Що таке JSON?
Відповідь: JSON (JavaScript Object Notation) - це легкочитний людиною текстовий формат обміну даними, побудований на парах ключ-значення та впорядкованих списках. Він є загальноприйнятим стандартом передачі структурованої інформації в сучасних REST API завдяки мовній незалежності.

Питання: Що таке REST?
Відповідь: REST (Representational State Transfer) - це архітектурний стиль побудови розподілених веб-сервісів, що спирається на стандарти HTTP, роботу з ресурсами через унікальні URI, модель без збереження стану (stateless) та стандартні методи взаємодії.

Питання: Що таке resource у REST?
Відповідь: Ресурс у REST - це будь-яка концептуальна сутність або об'єкт предметної області, до якого можна отримати доступ або яким можна маніпулювати через мережу за визначеним унікальним URI. Наприклад, окремий студент, курс чи список оцінок є ресурсами системи.

Питання: Що таке endpoint?
Відповідь: Endpoint (кінцева точка) - це конкретна комбінація URL-шляху та HTTP-методу у веб-сервісі, яка реалізує визначену дію над ресурсом. Клієнти звертаються до endpoint для читання або зміни даних за встановленим інтерфейсом.

Питання: Що таке path parameter?
Відповідь: Path parameter - це динамічна частина URL-шляху, яка виступає ідентифікатором конкретного ресурсу або його вкладеності (наприклад, {student_id} у маршруті /students/{student_id}). FastAPI автоматично витягує і валідує такі параметри як аргументи функції-обробника.

Питання: Що таке query parameter?
Відповідь: Query parameter - це необов'язковий іменований параметр у рядку запиту після знака питання (наприклад, ?limit=10&page=2), який використовується для фільтрації, сортування або пагінації ресурсів без зміни основного шляху.

Питання: Що таке request body?
Відповідь: Request body (тіло запиту) - це дані, які клієнт передає серверу в структурованому форматі (найчастіше JSON) для створення чи редагування ресурсу. Сервер розбирає тіло запиту відповідно до заголовка Content-Type і валідує його через схеми.

Питання: Що таке FastAPI?
Відповідь: FastAPI - це сучасний високопродуктивний асинхронний веб-фреймворк для Python, призначений для швидкої розробки REST API. Він базується на стандартах Starlette, Pydantic та OpenAPI, автоматично генеруючи інтерактивну документацію й виконуючи валідацію типів.

Питання: Для чого використовується Uvicorn?
Відповідь: Uvicorn - це швидкий ASGI-сервер (Asynchronous Server Gateway Interface) для Python, який запускає та обслуговує асинхронні веб-додатки на кшталт FastAPI. Він обробляє вхідні мережеві TCP-з'єднання й передає HTTP-запити в додаток через цикл подій.

Питання: Для чого потрібний Swagger UI?
Відповідь: Swagger UI потрібний для інтерактивної візуалізації та ручного тестування OpenAPI-документації REST API безпосередньо у веб-браузері. Він дозволяє розробникам переглядати наявні маршрути, моделі схем і виконувати реальні тестові запити до сервера.

Питання: Що таке Pydantic?
Відповідь: Pydantic - це бібліотека для Python, яка реалізує валідацію, серіалізацію та синтаксичний розбір даних на основі стандартних анотацій типів. Вона забезпечує приведення вхідних типів до очікуваних і генерує чіткі помилки у разі їхньої невідповідності.

Питання: Що таке validation?
Відповідь: Validation (валідація) - це процес автоматичної перевірки вхідних або вихідних даних на відповідність встановленим типам, обов'язковим полям, діапазонам значень та бізнес-правилам до початку їхньої обробки логікою програми.

Питання: Що таке response model?
Відповідь: Response model у FastAPI - це Pydantic-модель, яка декларує вихідну схему даних endpoint і використовується для автоматичної серіалізації, фільтрації приватних полів та генерування документації схеми відповіді.

Питання: Що таке synchronous programming?
Відповідь: Синхронне програмування - це підхід, при якому інструкції та виклики функцій виконуються послідовно один за одним. Під час тривалих операцій введення-виведення (I/O) поточний потік виконання блокується, очікуючи завершення передачі даних.

Питання: Що таке asynchronous programming?
Відповідь: Асинхронне програмування - це підхід, який дозволяє продовжувати виконання інших завдань у рамках одного потоку під час очікування операцій введення-виведення (I/O). Воно перемикає контекст у точках очікування, суттєво заощаджуючи ресурси процесора без паралельного виконання коду на CPU.

Питання: Для чого використовується async?
Відповідь: Ключове слово async використовується у Python для оголошення асинхронної функції (корутини). Воно сигналізує інтерпретатору, що всередині цієї функції можуть бути точки кооперативного перемикання контексту через оператор await.

Питання: Для чого використовується await?
Відповідь: Ключове слово await використовується для призупинення виконання поточної корутини до завершення асинхронної операції (awaitable-об'єкта) без блокування всього циклу подій. У цей час event loop може виконувати інші готові завдання.

Питання: Що таке coroutine?
Відповідь: Корутина (coroutine) - це спеціалізована функція, яка може призупиняти своє виконання і згодом відновлювати його зі збереженням внутрішнього стану. У Python вона створюється за допомогою виклику async def функції і для виконання має бути передана до event loop.

Питання: Що таке event loop?
Відповідь: Event loop (цикл подій) - це центральний механізм виконання асинхронного коду в asyncio, який керує чергою подій, відстежує готовність дескрипторів I/O і передає керування активним корутинам.

Питання: Що таке Task?
Відповідь: asyncio.Task - це обгортка навколо корутини, яка реєструє її в циклі подій для незалежного планування та конкурентного виконання. Вона успадковує клас Future і дозволяє відстежувати стан, скасовувати операцію або отримувати результат виконання.

Питання: Для чого використовується asyncio.create_task()?

Відповідь: Функція asyncio.create_task() використовується для планування виконання корутини у фоновому режимі в межах поточного event loop без необхідності негайно чекати її результату. Це дозволяє запускати конкурентні I/O-операції одночасно.

Питання: Для чого використовується asyncio.gather()?

Відповідь: Функція asyncio.gather() використовується для конкурентного запуску кількох awaitable-об'єктів або завдань та очікування завершення їх усіх із поверненням списку результатів у вихідному порядку.

Питання: Що таке timeout?

Відповідь: Timeout - це граничний інтервал часу, виділений на виконання операції, після перевищення якого операція примусово переривається (наприклад, через виняток TimeoutException). Він захищає систему від вічного зависання при збоях зовнішніх сервісів чи мережі.

Питання: Для чого потрібний retry?

Відповідь: Механізм retry потрібний для повторення операцій, що завершилися тимчасовою або транзитною мережевою помилкою (наприклад, затримкою зв'язку чи статусом 503). Це підвищує надійність і відмовостійкість розподіленої системи без необхідності ручного перезапуску користувачем.

Питання: Що таке backoff?

Відповідь: Backoff - це стратегія додавання зростаючої паузи перед кожною наступною спробою retry, найчастіше експоненційної (наприклад, 1с, 2с, 4с) із можливим випадковим зсувом (jitter). Вона запобігає перевантаженню відновлюваного сервера лавиною одночасних запитів.

Питання: Для чого використовується Semaphore?

Відповідь: asyncio.Semaphore використовується для обмеження кількості завдань, які можуть одночасно отримувати доступ до певного спільного ресурсу або виконувати I/O-операції. Він захищає пул з'єднань або зовнішній API від вичерпання лімітів і перевантаження.

Питання: Що таке HTTPX?

Відповідь: HTTPX - це сучасна повнофункціональна клієнтська HTTP-бібліотека для Python 3, яка надає як синхронний, так і асинхронний інтерфейси взаємодії. Вона підтримує HTTP/1.1 і HTTP/2, connection pooling та стандартний синтаксис на основі async/await.

Питання: Чим httpx.AsyncClient відрізняється від синхронного клієнта?

Відповідь: httpx.AsyncClient виконує мережеві запити неблокуючим способом за допомогою асинхронного введення-виведення під керуванням event loop. Синхронний клієнт блокує весь потік виконання на час очікування відповіді від сервера.

Питання: Чому AsyncClient бажано повторно використовувати?

Відповідь: Повторне використання AsyncClient через контекстний менеджер або довгоживучий екземпляр дозволяє зберігати відкритим пул TCP-з'єднань (connection pool) та повторно використовувати TLS-сесії. Це суттєво зменшує накладні витрати на встановлення нових з'єднань для кожного запиту.

Питання: Навіщо обмежувати паралелізм?

Відповідь: Паралелізм і конкурентність обмежують для того, щоб запобігти вичерпанню сокетів, файлових дескрипторів, пулів з'єднань із базою даних або перевищенню лімітів запитів (rate limits) сторонніх API. Надмірне навантаження призводить до падіння сервісів через виснаження системних ресурсів.

Питання: Чому не слід запускати тисячі tasks одночасно?

Відповідь: Запуск тисяч tasks одночасно створює надмірну кількість одночасних з'єднань, що призводить до помилок нестачі пам'яті (OOM), перевищення ліміту відкритих дескрипторів у системі та суттєвого перевантаження циклу подій.

Питання: Як REST API взаємодіє з Repository Layer?

Відповідь: REST API в обробниках маршрутів приймає валідовані запити, звертається до сервісного шару або напряму викликає методи Repository Layer для роботи зі сховищем даних, а отримані доменні сутності перетворює у відповідні Pydantic-моделі відповідей.

Питання: Як FastAPI працює з Pydantic?

Відповідь: FastAPI використовує Pydantic-моделі як анотації типів для параметрів функцій endpoint та атрибута response_model. На їхній основі фреймворк автоматично десеріалізує JSON, валідує вхідні поля, генерує схеми OpenAPI та серіалізує вихідні об'єкти.

Питання: Як протестувати REST API?

Відповідь: REST API тестують за допомогою спеціалізованих клієнтів, таких як TestClient або асинхронного httpx.AsyncClient у тестах pytest. Тести надсилають запити до реальних endpoints і перевіряють статус-коди, правильність заголовків та тіло поверненого JSON.

Питання: Чим API integration test відрізняється від unit test?

Відповідь: Unit test ізольовано перевіряє окрему функцію, метод чи клас із підміною зовнішніх залежностей тестовими дублерами (mock). API integration test перевіряє спільну роботу цілого ланцюжка компонентів - від HTTP-маршрутизатора й валідації Pydantic до бізнес-логіки та взаємодії зі сховищем даних.

Висновок:

У ході лабораторної роботи успішно реалізовано REST API на FastAPI для обліку студентів і навчальних груп із використанням Pydantic-схем та SQLAlchemy. Забезпечено надійне керування транзакціями із захистом від блокувань SQLite у конкурентному режимі, строгу валідацію граничних значень (включно з захистом від переповнення цілих чисел) та розроблено стійкий асинхронний клієнт із механізмами повторних спроб. Усі 126 автоматизованих тестів успішно пройдено, коректність ендпоінтів перевірено на реальному сервісі.